

Assignment

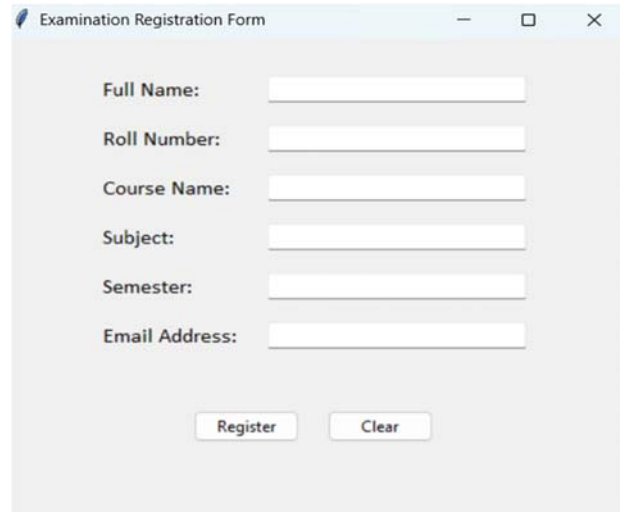
1. Design and implement a Python GUI application using Tkinter that enables new users to register by entering their personal details.

The GUI should include:

- Title: “Examination Registration Form”
- Labels and Entry Fields for the following details:
 - Full Name
 - Roll Number
 - Course Name
 - Subject
 - Semester
 - Email Address
- Buttons:
 - Register (to submit the form)
 - Clear (to reset all fields)

When the Register button is clicked:

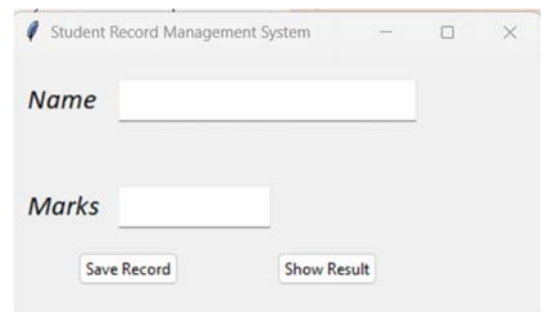
- Validate that no field is left empty.
- If all fields are filled:
 - Display a message box: “Registration Successful!”
- If any field is empty:
 - Display an error message: “All fields are required!”
 - When the Clear button is clicked:
- All entry fields should be cleared/reset.



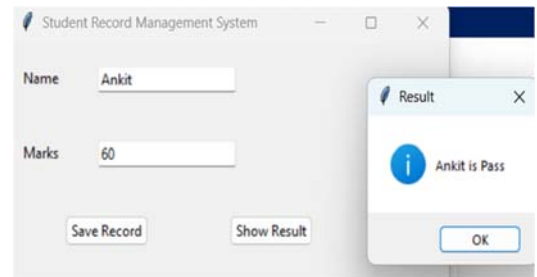
The screenshot shows a Tkinter window titled "Examination Registration Form". It contains six labeled entry fields: "Full Name:", "Roll Number:", "Course Name:", "Subject:", "Semester:", and "Email Address:". At the bottom of the window, there are two buttons: "Register" and "Clear".

2. Create a Student Record Management System using Tkinter GUI with the following features:

- Create a class Student:
 - Attribute: name, marks
 - Method to display result
 - If marks>40, Pass
 - Otherwise, fail
- GUI should have:
 - Entry field of name and marks
 - Buttons
 - Save Record
 - Show Result
- File Handling:
 - Save student data into a file (students.txt)
 - Append new records
- Exception Handling:
 - Handle invalid marks input (non-numeric)
 - Handle file errors

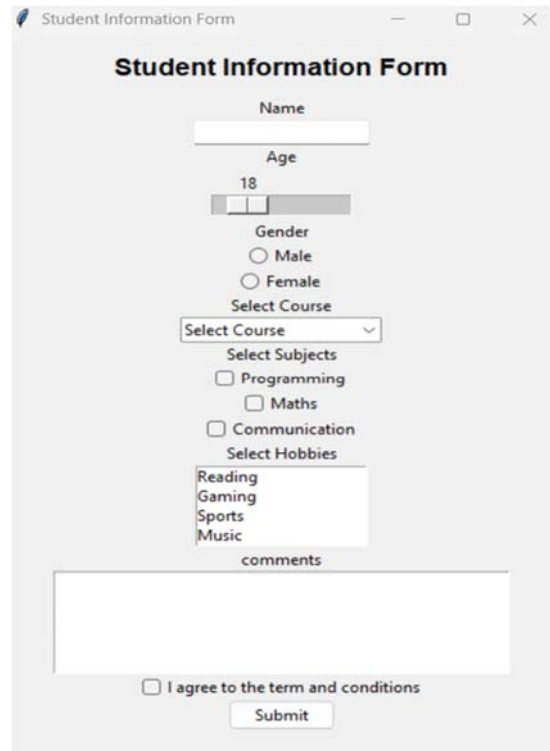


The screenshot shows a Tkinter window titled "Student Record Management System". It has two labeled entry fields: "Name" and "Marks". Below these fields are two buttons: "Save Record" and "Show Result".



This screenshot shows the same "Student Record Management System" window with "Name" set to "Ankit" and "Marks" set to "60". A small "Result" dialog box is open in the foreground, displaying a blue information icon, the text "Ankit is Pass", and an "OK" button.

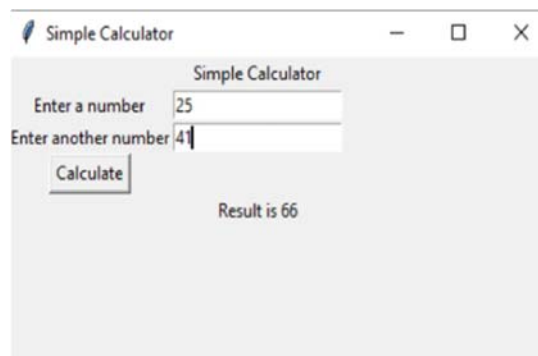
3. Create a user-friendly GUI application to collect and store student details using Tkinter as shown in figure. After clicking on the Submit button, save the entered data into a csv/txt file.



The screenshot shows a Tkinter window titled "Student Information Form". Inside, there's a form with the following fields and controls:

- Name:** A text input field.
- Age:** A spinbox with the value "18" selected.
- Gender:** Two radio buttons labeled "Male" and "Female".
- Select Course:** A dropdown menu with "Select Course" as the placeholder text.
- Select Subjects:** Three checkboxes labeled "Programming", "Maths", and "Communication".
- Select Hobbies:** A list box containing "Reading", "Gaming", "Sports", and "Music".
- comments:** A large text area for comments.
- Agreement:** A checkbox labeled "I agree to the term and conditions".
- Submit:** A button at the bottom right.

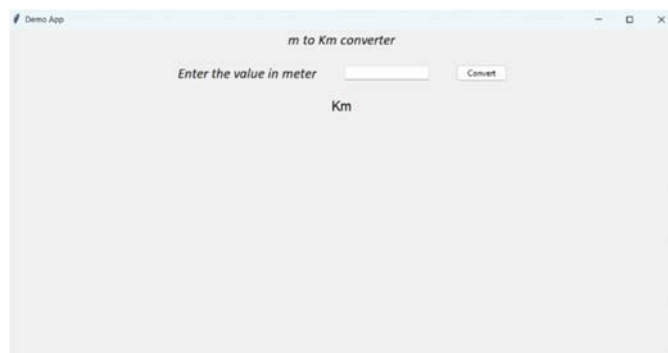
4. Develop a Python Tkinter application that takes two numbers as input, adds them, and displays the result.



The screenshot shows a Tkinter window titled "Simple Calculator". Inside, there's a form with the following fields and controls:

- Enter a number:** A text input field with the value "25".
- Enter another number:** A text input field with the value "4".
- Calculate:** A button.
- Result:** A label displaying "Result is 66".

5. Develop a Python Tkinter application that takes input in meters and converts it into Km.



The screenshot shows a Tkinter window titled "Demo App". Inside, there's a form with the following fields and controls:

- m to Km converter:** A label at the top.
- Enter the value in meter:** A text input field.
- Convert:** A button.
- Km:** A label below the input field.